PostgreSQL

Enterprise applications

NetApp
March 13, 2024

This PDF was generated from https://docs.netapp.com/de-de/ontap-apps-dbs/postgres/postgres-overview.html on March 13, 2024. Always check docs.netapp.com for the latest.

Inhalt

PostgreSQL . . . 1
PostgreSQL-Datenbanken auf ONTAP . . . 1
Datenbankkonfiguration . . . 1
Storage-Konfiguration . . . 6
Datensicherung . . . 9

PostgreSQL

PostgreSQL-Datenbanken auf ONTAP

PostgreSQL wird mit Varianten wie PostgreSQL, PostgreSQL Plus und EDB Postgres Advanced Server (EPAS) geliefert. PostgreSQL wird typischerweise als Back-End-Datenbank für Multi-Tier-Applikationen implementiert. Es wird von gängigen Middleware-Paketen (wie PHP, Java, Python, Tcl/TK, ODBC, Und JDBC) und war in der Vergangenheit eine beliebte Wahl für Open-Source-Datenbankmanagementsysteme. NetApp ONTAP ist eine ausgezeichnete Wahl für die Ausführung von PostgreSQL-Datenbanken aufgrund seiner Zuverlässigkeit, seiner hohen Performance und seiner effizienten Datenmanagementfunktionen.

> Diese Dokumentation zu ONTAP und der PostgreSQL-Datenbank ersetzt die zuvor veröffentlichte *TR-4770: PostgreSQL-Datenbank unter ONTAP Best Practices.*

Mit dem exponentiellen Datenwachstum wird das Datenmanagement für Unternehmen komplexer. Dadurch steigen die Lizenz-, Betriebs-, Support- und Wartungskosten. Zur Senkung der Gesamtbetriebskosten empfiehlt sich der Wechsel von kommerziellen zu Open-Source-Datenbanken mit zuverlässigem, leistungsstarkem Back-End Storage.

ONTAP ist die ideale Plattform, da ONTAP buchstäblich für Datenbanken entworfen ist. Es wurden speziell für die Anforderungen von Datenbank-Workloads zahlreiche Funktionen wie die Optimierung der zufälligen I/O-Latenz bis hin zur erweiterten Quality of Service (QoS) und grundlegenden FlexClone-Funktionalität erstellt.

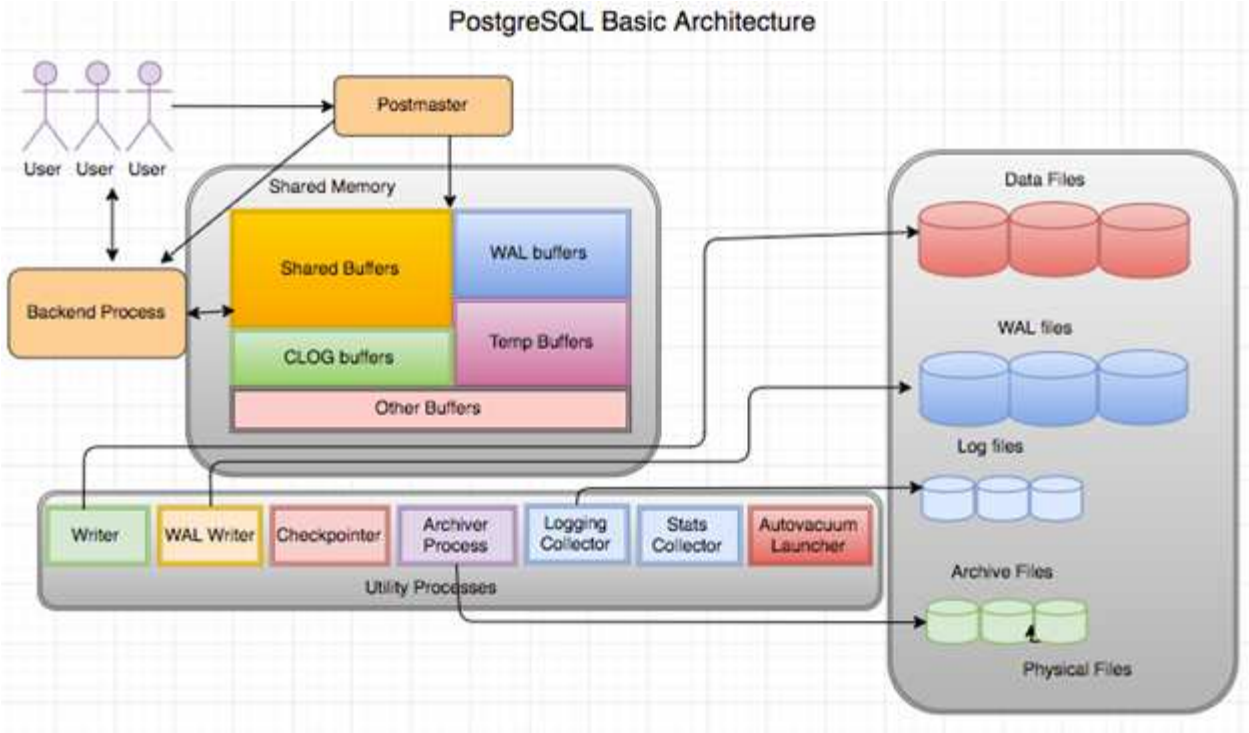
Zusätzliche Funktionen wie unterbrechungsfreie Upgrades (inkl. Storage-Austausch) stellen sicher, dass Ihre geschäftskritischen Datenbanken verfügbar bleiben. Darüber hinaus besteht die Möglichkeit zur sofortigen Disaster Recovery für große Umgebungen über MetroCluster oder zur Auswahl von Datenbanken mit SnapMirror Active Sync.

Am wichtigsten ist jedoch, dass ONTAP eine unübertroffene Performance sowie die Möglichkeit bietet, die Lösung entsprechend Ihren spezifischen Anforderungen zu dimensionieren. Unsere High-End-Systeme bieten über 1 Mio. IOPS bei Latenzen im Mikrosekundenbereich. Wenn Sie jedoch nur 100.000 IOPS benötigen, können Sie Ihre Storage-Lösung mit einem kleineren Controller dimensionieren, auf dem immer noch genau dasselbe Storage-Betriebssystem ausgeführt wird.

Datenbankkonfiguration

PostgreSQL Architektur

PostgreSQL ist ein RDBMS, das auf Client- und Serverarchitektur basiert. Eine PostgreSQL-Instanz wird als Datenbank-Cluster bezeichnet, bei dem es sich um eine Sammlung von Datenbanken und nicht um eine Sammlung von Servern handelt.



In einer PostgreSQL-Datenbank gibt es drei Hauptelemente: Den Postmaster, das Frontend (Client) und das Backend Der Client sendet Anfragen an den Postmaster mit Informationen wie IP-Protokoll und zu welcher Datenbank eine Verbindung hergestellt werden soll. Der Postmaster authentifiziert die Verbindung und leitet sie zur weiteren Kommunikation an den Back-End-Prozess weiter. Der Back-End-Prozess führt die Abfrage aus und sendet Ergebnisse direkt an das Frontend (Client).

Eine PostgreSQL-Instanz basiert auf einem Multiprocess-Modell statt auf einem Multithread-Modell. Es gibt mehrere Prozesse für verschiedene Jobs, und jeder Prozess hat seine eigene Funktionalität. Zu den wichtigsten Prozessen gehören der Clientprozess, der WAL Writer-Prozess, der Background Writer-Prozess und der Checkpointer-Prozess:

- Wenn ein Client-Prozess (Vordergrund) Lese- oder Schreibanforderungen an die PostgreSQL-Instanz sendet, werden keine Daten direkt auf die Festplatte geschrieben oder gelesen. Zuerst werden die Daten in gemeinsam genutzten Puffern und WAL-Puffern (Write-Ahead Logging) gepuffert.
- Ein WAL-Schreibprozess manipuliert den Inhalt der gemeinsam genutzten Puffer und WAL-Puffer, um in die WAL-Protokolle zu schreiben. WAL-Protokolle sind in der Regel Transaktionsprotokolle von PostgreSQL und werden sequenziell geschrieben. Um die Reaktionszeit aus der Datenbank zu verbessern, schreibt PostgreSQL zunächst in die Transaktionsprotokolle und bestätigt den Client.
- Um die Datenbank in einen konsistenten Zustand zu versetzen, überprüft der Background Writer-Prozess den gemeinsam genutzten Puffer regelmäßig auf fehlerhafte Seiten. Anschließend überträgt es die Daten auf die Datendateien, die auf NetApp Volumes oder LUNs gespeichert sind.
- Der Checkpointer-Prozess läuft auch periodisch (seltener als der Hintergrundprozess) und verhindert jegliche Änderung der Puffer. Er signalisiert dem WAL Writer-Prozess, den Checkpoint-Datensatz zu schreiben und an das Ende der WAL-Protokolle zu löschen, die auf der NetApp-Festplatte gespeichert sind. Er signalisiert auch, dass der Background Writer-Prozess alle fehlerhaften Seiten auf die Festplatte schreibt und auf diese schreibt.

PostgreSQL-Initialisierungsparameter

Sie erstellen mithilfe von ein neues Datenbankcluster `initdb` Programm. An `initdb`

Skript erstellt die Datendateien, Systemtabellen und Vorlagendatenbanken (template0 und template 1), die den Cluster definieren.

Die Vorlagendatenbank stellt eine Bestandsdatenbank dar. Es enthält Definitionen für Systemtabellen, Standardansichten, Funktionen und Datentypen. `pgdata` Fungiert als Argument für den `initdb` Skript, das den Speicherort des Datenbank-Clusters angibt.

Alle Datenbankobjekte in PostgreSQL werden intern von den jeweiligen OIDs verwaltet. Tabellen und Indizes werden auch von einzelnen OIDs verwaltet. Die Beziehungen zwischen Datenbankobjekten und ihren jeweiligen OIDs werden je nach Objekttyp in entsprechenden Systemkatalogtabellen gespeichert. OIDs von Datenbanken und Heap-Tabellen werden beispielsweise in gespeichert `pg_database` Und `pg_class. Sie können die OIDs durch Abfragen auf dem PostgreSQL-Client ermitteln.

Jede Datenbank verfügt über eigene einzelne Tabellen und Indexdateien, die auf 1 GB beschränkt sind. Jede Tabelle hat zwei zugehörige Dateien, die jeweils mit dem Suffix versehen sind `_fsm` Und `_vm`. Sie werden als freie Raumkarte und Sichtbarkeitskarte bezeichnet. Diese Dateien speichern die Informationen über die freie Speicherkapazität und haben Sichtbarkeit auf jeder Seite in der Tabellendatei. Indizes verfügen nur über individuelle freie Speicherplatzkarten und haben keine Sichtbarkeits-Karten.

Der `pg_xlog/pg_wal` Das Verzeichnis enthält die Write-Ahead-Protokolle. Mit Write-Ahead-Protokollen werden die Zuverlässigkeit und Performance der Datenbank verbessert. Immer wenn Sie eine Zeile in einer Tabelle aktualisieren, schreibt PostgreSQL die Änderung zuerst in das Write-Ahead-Protokoll und schreibt später die Änderungen auf die eigentlichen Datenseiten auf eine Festplatte. Der `pg_xlog` Das Verzeichnis enthält normalerweise mehrere Dateien, aber initdb erstellt nur die erste. Zusätzliche Dateien werden bei Bedarf hinzugefügt. Jede xlog-Datei ist 16 MB lang.

Konfiguration der PostgreSQL-Datenbank mit ONTAP

Es gibt mehrere PostgreSQL-Tuning-Konfigurationen, die die Performance verbessern können.

Die am häufigsten verwendeten Parameter sind:

- `max_connections = <num>`: Die maximale Anzahl von Datenbankverbindungen, die gleichzeitig verfügbar sind. Verwenden Sie diesen Parameter, um den Austausch auf Festplatte zu beschränken und die Leistung zu unterbinden. Je nach Anwendungsanforderung können Sie diesen Parameter auch für die Einstellungen des Verbindungspools anpassen.
- `shared_buffers = <num>`: Die einfachste Methode zur Verbesserung der Leistung Ihres Datenbankservers. Die Standardeinstellung ist niedrig für die meisten modernen Hardware. Sie wird während der Bereitstellung auf ca. 25 % des verfügbaren RAM auf dem System eingestellt. Diese Parametereinstellung hängt davon ab, wie sie mit bestimmten Datenbankinstanzen funktioniert; Sie müssen die Werte möglicherweise durch Versuch und Fehler erhöhen oder verringern. Bei einer hohen Einstellung wird die Performance jedoch wahrscheinlich beeinträchtigt.
- `effective_cache_size = <num>`: Dieser Wert teilt PostgreSQL's Optimizer mit, wie viel Speicher PostgreSQL für das Caching von Daten zur Verfügung hat und hilft bei der Bestimmung, ob ein Index verwendet werden soll. Ein größerer Wert erhöht die Wahrscheinlichkeit, einen Index zu verwenden. Dieser Parameter sollte auf die Größe des zugewiesenen Speichers eingestellt werden `shared_buffers` Und die Menge an verfügbarem BS-Cache. Dieser Wert liegt häufig bei mehr als 50 % des gesamten Systemspeichers.
- `work_mem = <num>`: Dieser Parameter steuert die Speichermenge, die in Sort-Operationen und Hash-Tabellen verwendet werden soll. Wenn Sie eine starke Sortierung in Ihrer Anwendung ausführen, müssen Sie möglicherweise den Speicherplatz erhöhen, aber seien Sie vorsichtig. Es handelt sich nicht um einen

systemweiten Parameter, sondern um einen pro-Operation-Parameter. Wenn eine komplexe Abfrage mehrere Sortieroperationen enthält, verwendet sie mehrere Work_mem-Speichereinheiten, und mehrere Back-Ends könnten dies gleichzeitig tun. Diese Abfrage kann oft dazu führen, dass Ihr Datenbankserver ausgetauscht wird, wenn der Wert zu groß ist. Diese Option wurde zuvor in älteren PostgreSQL-Versionen als sort_mem bezeichnet.

- `fsync = <boolean> (on or off)`: Dieser Parameter legt fest, ob alle WAL-Seiten mit fsync() synchronisiert werden sollen, bevor eine Transaktion durchgeführt wird. Wenn Sie sie deaktivieren, kann die Schreibleistung manchmal verbessert werden, und wenn Sie sie einschalten, erhöht sich der Schutz vor dem Risiko von Beschädigungen, wenn das System abstürzt.
- `checkpoint_timeout`: Der Checkpoint-Prozess überträgt die Daten auf die Festplatte. Dies beinhaltet viele Lese-/Schreibvorgänge auf der Festplatte. Der Wert wird in Sekunden festgelegt, und niedrigere Werte verringern die Absturzwiederherstellungszeit, und höhere Werte können die Belastung der Systemressourcen verringern, indem die Checkpoint-Anrufe reduziert werden. Legen Sie je nach Wichtigkeit der Anwendung, Auslastung und Verfügbarkeit der Datenbank den Wert von Checkpoint_Timeout fest.
- `commit_delay = <num>` Und `commit_siblings = <num>`: Diese Optionen werden zusammen verwendet, um die Leistung zu verbessern, indem mehrere Transaktionen, die auf einmal begehen, ausgeschrieben werden. Wenn mehrere commit_Geschwister-Objekte aktiv sind, sobald Ihre Transaktion abgeschlossen ist, wartet der Server auf commit_delay Mikrosekunden, um zu versuchen, mehrere Transaktionen gleichzeitig zu begehen.
- `max_worker_processes / max_parallel_workers`: Konfigurieren Sie die optimale Anzahl von Arbeitern für Prozesse. Max_Parallel_Workers entspricht der Anzahl der verfügbaren CPUs. Je nach Anwendungsdesign erfordern Abfragen möglicherweise weniger Mitarbeiter für parallele Vorgänge. Es ist besser, den Wert für beide Parameter gleich zu halten, aber den Wert nach dem Testen anzupassen.
- `random_page_cost = <num>`: Dieser Wert steuert die Art und Weise, wie PostgreSQL nicht-sequentielle Datenträger liest. Ein höherer Wert bedeutet, dass PostgreSQL eher einen sequenziellen Scan anstelle eines Indexscans verwendet, was darauf hinweist, dass Ihr Server über schnelle Festplatten verfügt.Ändern Sie diese Einstellung, nachdem Sie andere Optionen wie planbasierte Optimierung, Staubsaugen, Indexieren auf Abfragen oder Schema überprüft haben.
- `effective_io_concurrency = <num>`: Dieser Parameter legt die Anzahl der gleichzeitigen Festplatten-I/O-Operationen fest, die PostgreSQL gleichzeitig auszuführen versucht. Wenn Sie diesen Wert erhöhen, erhöht sich die Anzahl der I/O-Vorgänge, die jede einzelne PostgreSQL-Sitzung parallel initiieren möchte. Der zulässige Bereich ist 1 bis 1,000 oder Null, um die Ausgabe asynchroner I/O-Anfragen zu deaktivieren. Derzeit wirkt sich diese Einstellung nur auf Bitmap-Heap-Scans aus. Solid State Drives (SSDs) und anderer speicherbasierter Storage (NVMe) können oft zahlreiche gleichzeitige Anforderungen verarbeiten, sodass der beste Nutzen aus Hunderten von Laufwerken zu ziehen ist.

Eine vollständige Liste der PostgreSQL-Konfigurationsparameter finden Sie in der PostgreSQL-Dokumentation.

TOAST

TOAST steht für die Oversized-Attribute Storage-Technik. PostgreSQL verwendet eine feste Seitengröße (üblicherweise 8 KB) und erlaubt nicht, dass Tupel mehrere Seiten umfassen. Daher ist es nicht möglich, große Feldwerte direkt zu speichern. Wenn Sie versuchen, eine Zeile zu speichern, die diese Größe überschreitet, bricht TOAST die Daten großer Spalten in kleinere „Stücke“ und speichert sie in einem TOAST Tisch.

Die großen Werte der getoasteten Attribute werden nur dann herausgezogen (wenn sie überhaupt ausgewählt sind), wenn der Ergebnissatz an den Client gesendet wird. Die Tabelle selbst ist viel kleiner und kann mehr Zeilen in den gemeinsam genutzten Puffer-Cache passen als ohne Out-of-Line Storage (TOAST).

VAKUUM

Im normalen PostgreSQL-Betrieb werden Tupel, die durch eine Aktualisierung gelöscht oder veraltet gemacht werden, nicht physisch aus ihrer Tabelle entfernt; sie bleiben vorhanden, bis VAKUUM ausgeführt wird. Daher müssen Sie regelmäßig VAKUUM betreiben, insbesondere auf häufig aktualisierten Tabellen. Der belegte Speicherplatz muss dann zur Wiederverwendung durch neue Zeilen zurückgewonnen werden, um einen Ausfall von Festplattenspeicher zu vermeiden. Er gibt jedoch nicht den Speicherplatz an das Betriebssystem zurück.

Der freie Platz innerhalb einer Seite ist nicht fragmentiert. VACUUM schreibt den gesamten Block neu, verpackt die restlichen Zeilen und hinterlässt einen einzigen zusammenhängenden Block freien Speicherplatz auf einer Seite.

Dagegen verdichtet VACUUM FULL Tabellen aktiv, indem eine völlig neue Version der Tabellendatei ohne Totraum geschrieben wird. Diese Aktion minimiert die Größe des Tisches, kann aber lange dauern. Außerdem wird zusätzlicher Speicherplatz für die neue Kopie der Tabelle benötigt, bis der Vorgang abgeschlossen ist. Ziel des routinemäßigen VAKUUMS ist es, die VOLLE VAKUUMAKTIVITÄT zu vermeiden. Bei diesem Prozess werden nicht nur Tabellen auf der Mindestgröße gespeichert, sondern auch der Festplattenspeicherplatz weiterhin gleichmäßig genutzt.

PostgreSQL-Tablespaces

Bei der Initialisierung des Datenbank-Clusters werden automatisch zwei Tablespaces erstellt.

Der `pg_global` Tablespace wird für freigegebene Systemkataloge verwendet. Der `pg_default` Tablespace ist der Standard-Tablespace der Datenbanken temple1 und template0. Wenn die Partition oder das Volume, auf der das Cluster initialisiert wurde, nicht mehr genügend Speicherplatz hat und nicht erweitert werden kann, kann ein Tablespace auf einer anderen Partition erstellt und verwendet werden, bis das System neu konfiguriert werden kann.

Ein stark genutzter Index kann auf einer schnellen, hochverfügbaren Festplatte wie einem Solid-State-Gerät platziert werden. Darüber hinaus kann eine Tabelle mit archivierten Daten, die selten verwendet oder nicht Performance-kritisch sind, auf einem kostengünstigeren, langsameren Festplattensystem wie SAS- oder SATA-Laufwerken gespeichert werden.

Tablespaces sind Bestandteil des Datenbank-Clusters und können nicht als eigenständige Erfassung von Datendateien behandelt werden. Sie sind von Metadaten im Hauptdatenverzeichnis abhängig und können daher nicht an einen anderen Datenbankcluster angeschlossen oder einzeln gesichert werden. Wenn Sie einen Tablespace verlieren (durch Dateilöschung, Festplattenfehler usw.), kann der Datenbankcluster möglicherweise unlesbar werden oder nicht mehr starten. Wenn ein Tablespace auf einem temporären Dateisystem wie einer RAM-Festplatte platziert wird, besteht die Gefahr, dass der gesamte Cluster zuverlässig ist.

Nach der Erstellung kann ein Tablespace aus jeder beliebigen Datenbank verwendet werden, wenn der anfordernde Benutzer über ausreichende Berechtigungen verfügt. PostgreSQL verwendet symbolische Links, um die Implementierung von Tablespaces zu vereinfachen. PostgreSQL fügt dem eine Zeile hinzu `pg_tablespace` Tabelle (eine clusterwide-Tabelle) und weist dieser Zeile eine neue Objektkennung (OID) zu. Schließlich verwendet der Server die OID, um einen symbolischen Link zwischen Ihrem Cluster und dem angegebenen Verzeichnis zu erstellen. Das Verzeichnis `$PGDATA/pg_tblspc` Enthält symbolische Links, die auf jeden nicht integrierten Tablespace verweisen, der im Cluster definiert ist.

Storage-Konfiguration

PostgreSQL-Datenbanken mit NFS-Dateisystemen

PostgreSQL-Datenbanken können auf NFSv3- oder NFSv4-Dateisystemen gehostet werden. Die beste Option hängt von Faktoren außerhalb der Datenbank ab.

Beispielsweise könnte das Sperrverhalten von NFSv4 in bestimmten Cluster-Umgebungen vorzuziehen sein. (Siehe "Hier" Für weitere Details)

Ansonsten sollte die Datenbankfunktionalität, einschließlich der Performance, nahezu identisch sein. Die einzige Voraussetzung ist die Verwendung des `hard` Mount-Option. Dies ist erforderlich, um sicherzustellen, dass weiche Timeouts keine nicht behebbaren E/A-Fehler verursachen.

Wenn NFSv4 als Protokoll gewählt wird, empfiehlt NetApp die Verwendung von NFSv4.1. Es gibt einige funktionale Verbesserungen am NFSv4.1-Protokoll, die die Ausfallsicherheit gegenüber NFSv4.0 verbessern.

Verwenden Sie die folgenden Mount-Optionen für allgemeine Datenbank-Workloads:

```
rw,hard,nointr,bg,vers=[3|4],proto=tcp,rsize=65536,wsize=65536
```

Wenn sequenzielle I/O-Vorgänge mit hohem I/O-Wert zu erwarten sind, kann die NFS-Übertragungsgröße wie im folgenden Abschnitt beschrieben erhöht werden.

NFS-Übertragungsgrößen

Standardmäßig beschränkt ONTAP die NFS-I/O-Größe auf 64K.

Zufälliger I/O mit den meisten Applikationen und Datenbanken verwendet eine viel kleinere Blockgröße, die weit unter dem 64K-Maximum liegt. Der I/O großer Blöcke wird in der Regel parallelisiert, sodass die 64K-Maximalgröße auch keine Einschränkung für die Erzielung der maximalen Bandbreite darstellt.

Es gibt einige Workloads, bei denen das 64K-Maximum eine Einschränkung darstellt. Insbesondere Vorgänge in einem einzigen Thread, wie Backup- oder Recovery-Vorgänge oder ein vollständiger Tabellenscan in einer Datenbank, laufen schneller und effizienter, wenn die Datenbank weniger, aber größere I/OS ausführen kann. Die optimale I/O-Handhabungsgröße für ONTAP beträgt 256 KB.

Die maximale Übertragungsgröße für eine bestimmte ONTAP SVM kann wie folgt geändert werden:

```
Cluster01::> set advanced
Warning: These advanced commands are potentially dangerous; use them only
when directed to do so by NetApp personnel.
Do you want to continue? {y|n}: y
Cluster01::*> nfs server modify -vserver vserver1 -tcp-max-xfer-size
262144
Cluster01::*>
```

Achtung
Verringern Sie niemals die maximal zulässige Übertragungsgröße auf ONTAP unter den Wert rsize/wsize der aktuell gemounteten NFS-Dateisysteme. Dies kann bei einigen Betriebssystemen zu Hängebleiben oder sogar Datenbeschädigungen führen. Wenn beispielsweise NFS-Clients derzeit auf 65536 rsize/wsize gesetzt sind, dann könnte die maximale Übertragungsgröße für ONTAP ohne Auswirkung auf die Clients selbst begrenzt werden, zwischen 65536 und 1048576 angepasst werden. Wenn Sie die maximale Übertragungsgröße unter 65536 verringern, können die Verfügbarkeit oder die Daten beeinträchtigt werden.

Sobald die Übertragungsgröße auf ONTAP-Ebene erhöht wurde, werden die folgenden Mount-Optionen verwendet:

```
rw,hard,nointr,bg,vers=[3|4],proto=tcp,rsize=262144,wsize=262144
```

NFSv3 TCP-Slot-Tabellen

Wenn NFSv3 mit Linux verwendet wird, ist es wichtig, die TCP-Slot-Tabellen ordnungsgemäß festzulegen.

TCP-Slot-Tabellen sind das NFSv3 Äquivalent zur Warteschlangentiefe des Host Bus Adapters (HBA). Diese Tabellen steuern die Anzahl der NFS-Vorgänge, die zu einem beliebigen Zeitpunkt ausstehen können. Der Standardwert ist normalerweise 16, was für eine optimale Performance viel zu niedrig ist. Das entgegengesetzte Problem tritt auf neueren Linux-Kerneln auf, die automatisch die Begrenzung der TCP-Slot-Tabelle auf ein Niveau erhöhen können, das den NFS-Server mit Anforderungen sättigt.

Um eine optimale Performance zu erzielen und Performance-Probleme zu vermeiden, passen Sie die Kernel-Parameter an, die die TCP-Slot-Tabellen steuern.

Führen Sie die aus `sysctl -a | grep tcp.*.slot_table` Und beobachten Sie die folgenden Parameter:

```
# sysctl -a | grep tcp.*.slot_table
sunrpc.tcp_max_slot_table_entries = 128
sunrpc.tcp_slot_table_entries = 128
```

Alle Linux-Systeme sollten enthalten `sunrpc.tcp_slot_table_entries`, Aber nur einige enthalten `sunrpc.tcp_max_slot_table_entries`. Beide sollten auf 128 gesetzt werden.

Achtung
Wenn diese Parameter nicht eingestellt werden, kann dies erhebliche Auswirkungen auf die Leistung haben. In einigen Fällen ist die Performance eingeschränkt, da das linux-Betriebssystem nicht genügend I/O ausgibt In anderen Fällen erhöht sich die I/O-Latenz, wenn das linux Betriebssystem versucht, mehr I/O-Vorgänge auszustellen, als gewartet werden kann.

PostgreSQL mit SAN-Dateisystemen

PostgreSQL-Datenbanken mit SAN werden in der Regel auf xfs-Dateisystemen gehostet, aber andere können verwendet werden, wenn sie vom OS-Anbieter unterstützt werden

Während eine einzelne LUN in der Regel bis zu 100.000 IOPS unterstützen kann, erfordern IO-intensive

Datenbanken in der Regel die Verwendung von LVM mit Striping.

LVM-Striping

Vor der Ära der Flash-Laufwerke wurde Striping verwendet, um die Performance-Einschränkungen rotierender Laufwerke zu überwinden. Beispiel: Wenn ein Betriebssystem einen Lesevorgang von 1 MB ausführen muss, würde das Lesen dieser 1 MB Daten von einem einzigen Laufwerk viel Festplattenkopf erfordern, der sucht und liest, da die 1 MB langsam übertragen wird. Wenn diese 1 MB Daten über 8 LUNs verteilt wurden, kann das Betriebssystem acht 128K-Lesevorgänge parallel ausführen und die für die 1-MB-Übertragung erforderliche Zeit verringern.

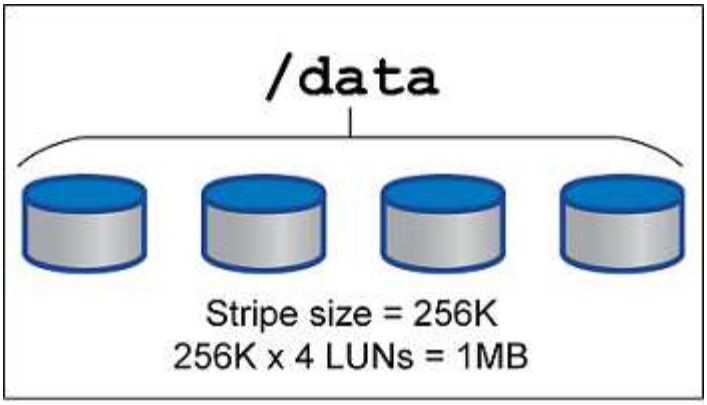
Das Striping mit rotierenden Laufwerken war schwieriger, da das I/O-Muster bereits im Vorfeld bekannt sein musste. Wenn das Striping nicht richtig auf die wahren I/O-Muster abgestimmt wurde, können Striping-Konfigurationen die Performance beeinträchtigen. Bei Oracle Datenbanken und insbesondere bei All-Flash-Konfigurationen ist Striping einfacher zu konfigurieren und hat sich nachweislich für eine drastische Verbesserung der Performance bewährt.

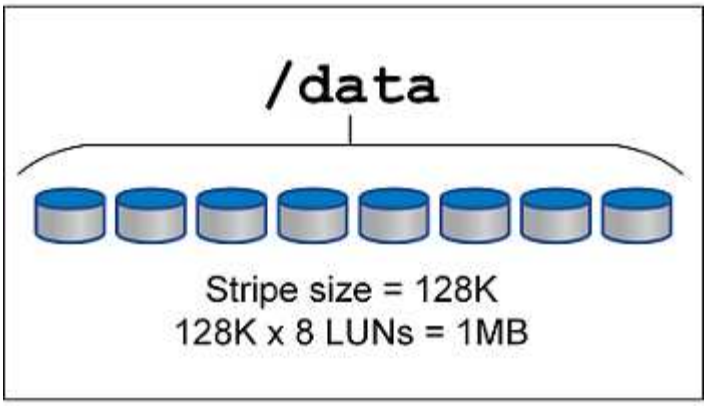
Logische Volume-Manager wie Oracle ASM Stripe sind standardmäßig aktiviert, aber native OS LVM nicht. Einige von ihnen verbinden mehrere LUNs als verkettete Geräte. Dies führt zu Datendateien, die auf einem und nur einem LUN-Gerät vorhanden sind. Dies verursacht Hotspots. Andere LVM-Implementierungen sind standardmäßig auf verteilte Extents eingestellt. Das ist ähnlich wie Striping, aber es ist gröber. Die LUNs in der Volume-Gruppe werden in große Teile geteilt, die als Extents bezeichnet werden und in der Regel in vielen Megabyte gemessen werden. Die logischen Volumes werden dann über diese Extents verteilt. Das Ergebnis ist ein zufälliger I/O-Vorgang für eine Datei, der auf LUNs verteilt werden sollte. Sequenzielle I/O-Vorgänge sind jedoch nicht so effizient wie möglich.

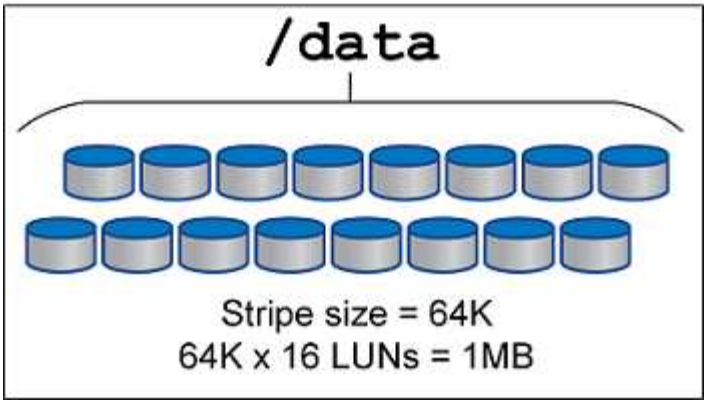
Die Performance-intensiven Applikations-I/O-Vorgänge erfolgen fast immer entweder (a) in Einheiten der grundlegenden Blockgröße oder (b) in Megabyte.

Das primäre Ziel einer Striped-Konfiguration ist es, sicherzustellen, dass Single-File I/O als eine Einheit ausgeführt werden kann. Multiblock-I/O, die eine Größe von 1 MB haben sollte, kann gleichmäßig über alle LUNs im Striped Volume hinweg parallelisiert werden. Das bedeutet, dass die Stripe-Größe nicht kleiner als die Blockgröße der Datenbank sein darf und die Stripe-Größe multipliziert mit der Anzahl der LUNs 1 MB betragen sollte.

Die folgende Abbildung zeigt drei mögliche Optionen für die Stripe-Größe und Breitenabstimmung. Die Anzahl der LUNs wird ausgewählt, um die oben beschriebenen Performance-Anforderungen zu erfüllen. In allen Fällen beträgt die Gesamtzahl der Daten innerhalb eines einzigen Stripes jedoch 1 MB.







Datensicherung

PostgreSQL Datensicherung

Einer der wichtigsten Aspekte des Storage-Designs ist die Sicherung von PostgreSQL Volumes. Kunden können ihre PostgreSQL-Datenbanken entweder mithilfe des Dump-Ansatzes oder mit Dateisystem-Backups sichern. In diesem Abschnitt werden die verschiedenen Ansätze zur Sicherung einzelner Datenbanken oder des gesamten Clusters erläutert.

Es gibt drei Ansätze für die Sicherung von PostgreSQL-Daten:

- SQL Server Dump
- Backup auf Filesystem-Ebene

- Kontinuierliche Archivierung

Die Idee hinter der SQL Server Dump-Methode besteht darin, eine Datei mit SQL Server-Befehlen zu generieren, die, wenn sie an den Server zurückgegeben wird, die Datenbank so neu erstellen kann, wie sie zum Zeitpunkt des Dump war. PostgreSQL stellt die Dienstprogramme zur Verfügung `pg_dump` Und `pg_dump_all` Zur Erstellung von individuellen Backups und Backups auf Cluster-Ebene. Diese Dumps sind logisch und enthalten nicht genügend Informationen, die von WAL Replay verwendet werden können.

Eine alternative Backup-Strategie ist die Verwendung von Backup auf Dateisystem-Ebene, bei der Administratoren direkt kopieren die Dateien, die PostgreSQL verwendet, um die Daten in der Datenbank zu speichern. Diese Methode erfolgt im Offline-Modus: Die Datenbank oder das Cluster muss heruntergefahren werden. Eine weitere Alternative ist die Verwendung `pg_basebackup` Zum Ausführen von Hot-Streaming-Backups der PostgreSQL-Datenbank.

PostgreSQL-Datenbanken und Storage Snapshots

Snapshot-basierte Backups mit PostgreSQL erfordern die Konfiguration von Snapshots für Datendateien, WAL-Dateien und archivierte WAL-Dateien, um eine vollständige oder zeitpunktgenaue Recovery zu ermöglichen.

Bei PostgreSQL-Datenbanken liegt die durchschnittliche Backup-Zeit mit Snapshots im Bereich von wenigen Sekunden bis zu wenigen Minuten. Diese Backup-Geschwindigkeit ist 60 bis 100 Mal schneller als `pg_basebackup` Und anderen Filesystem-basierten Backup-Ansätzen.

Snapshots auf NetApp Storage können sowohl ausfallkonsistent als auch applikationskonsistent sein. Ein Crash-konsistenter Snapshot wird auf dem Storage erstellt, ohne die Datenbank stillzustehen. Während sich die Datenbank im Backup-Modus befindet, wird ein applikationskonsistenter Snapshot erstellt. NetApp sorgt außerdem dafür, dass nachfolgende Snapshots dauerhaft inkrementelle Backups sind, um die Storage-Einsparungen und die Netzwerkeffizienz zu erhöhen.

Da Snapshots schnell sind und die System-Performance nicht beeinträchtigen, können Sie mehrere Snapshots täglich planen, anstatt wie bei anderen Streaming-Backup-Technologien täglich ein einziges Backup zu erstellen. Wenn ein Wiederherstellungs- und Wiederherstellungsvorgang erforderlich ist, verringert sich die Systemausfallzeit um zwei wichtige Funktionen:

- Dank der NetApp SnapRestore Datenwiederherstellungs-Technologie erfolgt die Wiederherstellung in Sekundenschnelle.
- Durch aggressive Recovery Point Objectives (RPOs) müssen weniger Datenbankprotokolle angewendet werden und auch die Recovery wird beschleunigt.

Für das Backup von PostgreSQL müssen Sie sicherstellen, dass die Datenvolumes gleichzeitig mit (Consistency Group) WAL und den archivierten Protokollen geschützt sind. Stellen Sie beim Kopieren von WAL-Dateien mit der Snapshot-Technologie sicher, dass Sie ausgeführt werden `pg_stop` Um alle WAL-Einträge zu löschen, die archiviert werden müssen. Wenn Sie die WAL-Einträge während der Wiederherstellung löschen, müssen Sie nur die Datenbank anhalten, das vorhandene Datenverzeichnis aufheben oder löschen und einen SnapRestore-Vorgang auf dem Speicher ausführen. Nachdem die Wiederherstellung abgeschlossen ist, können Sie das System mounten und in den aktuellen Status zurückversetzen. Für Point-in-Time Recovery können Sie WAL wiederherstellen und Protokolle archivieren. PostgreSQL entscheidet dann über den konsistentesten Punkt und stellt ihn automatisch wieder her.

Konsistenzgruppen sind in ONTAP eine Funktion, die ebenfalls empfohlen werden, wenn mehrere Volumes in eine einzelne Instanz oder in eine Datenbank mit mehreren Tablespaces gemountet sind. Ein Snapshot einer Konsistenzgruppe stellt sicher, dass alle Volumes gruppiert und geschützt sind. Eine Konsistenzgruppe kann

über den ONTAP System Manager effizient gemanagt werden und Sie können sie sogar klonen, um eine Instanzkopie einer Datenbank zu Test- oder Entwicklungszwecken zu erstellen.

Weitere Informationen zu Konsistenzgruppen finden Sie im "Übersicht über NetApp Konsistenzgruppen".

PostgreSQL Datensicherungssoftware

Das NetApp SnapCenter Plug-in für die PostgreSQL Datenbank bietet in Kombination mit Snapshot und NetApp FlexClone Technologien folgende Vorteile:

- Schnelles Backup und Restore:
- Platzsparende Klone:
- Aufbau eines schnellen und effektiven Disaster Recovery-Systems

> Unter den folgenden Umständen bevorzugen Sie die Premium-Backup-Partner von NetApp, z. B. Veeam Software und CommVault:
>
> - Management von Workloads in heterogener Umgebung
> - Speichern von Backups in der Cloud oder auf Tape zur langfristigen Aufbewahrung
> - Unterstützung für eine Vielzahl von Betriebssystemversionen und -Typen

SnapCenter Plugin für PostgreSQL ist Community-unterstütztes Plugin und das Setup und die Dokumentation ist auf NetApp Automation Store verfügbar. Mit SnapCenter können Anwender Datenbanken sichern sowie Daten Remote klonen und wiederherstellen.

Copyright-Informationen

Copyright © 2024 NetApp. Alle Rechte vorbehalten. Gedruckt in den USA. Dieses urheberrechtlich geschützte Dokument darf ohne die vorherige schriftliche Genehmigung des Urheberrechtsinhabers in keiner Form und durch keine Mittel – weder grafische noch elektronische oder mechanische, einschließlich Fotokopieren, Aufnehmen oder Speichern in einem elektronischen Abrufsystem – auch nicht in Teilen, vervielfältigt werden.

Software, die von urheberrechtlich geschütztem NetApp Material abgeleitet wird, unterliegt der folgenden Lizenz und dem folgenden Haftungsausschluss:

DIE VORLIEGENDE SOFTWARE WIRD IN DER VORLIEGENDEN FORM VON NETAPP ZUR VERFÜGUNG GESTELLT, D. H. OHNE JEGLICHE EXPLIZITE ODER IMPLIZITE GEWÄHRLEISTUNG, EINSCHLIESSLICH, JEDOCH NICHT BESCHRÄNKT AUF DIE STILLSCHWEIGENDE GEWÄHRLEISTUNG DER MARKTGÄNGIGKEIT UND EIGNUNG FÜR EINEN BESTIMMTEN ZWECK, DIE HIERMIT AUSGESCHLOSSEN WERDEN. NETAPP ÜBERNIMMT KEINERLEI HAFTUNG FÜR DIREKTE, INDIREKTE, ZUFÄLLIGE, BESONDERE, BEISPIELHAFTE SCHÄDEN ODER FOLGESCHÄDEN (EINSCHLIESSLICH, JEDOCH NICHT BESCHRÄNKT AUF DIE BESCHAFFUNG VON ERSATZWAREN ODER -DIENSTLEISTUNGEN, NUTZUNGS-, DATEN- ODER GEWINNVERLUSTE ODER UNTERBRECHUNG DES GESCHÄFTSBETRIEBS), UNABHÄNGIG DAVON, WIE SIE VERURSACHT WURDEN UND AUF WELCHER HAFTUNGSTHEORIE SIE BERUHEN, OB AUS VERTRAGLICH FESTGELEGTER HAFTUNG, VERSCHULDENSUNABHÄNGIGER HAFTUNG ODER DELIKTSHAFTUNG (EINSCHLIESSLICH FAHRLÄSSIGKEIT ODER AUF ANDEREM WEGE), DIE IN IRGENDEINER WEISE AUS DER NUTZUNG DIESER SOFTWARE RESULTIEREN, SELBST WENN AUF DIE MÖGLICHKEIT DERARTIGER SCHÄDEN HINGEWIESEN WURDE.

NetApp behält sich das Recht vor, die hierin beschriebenen Produkte jederzeit und ohne Vorankündigung zu ändern. NetApp übernimmt keine Verantwortung oder Haftung, die sich aus der Verwendung der hier beschriebenen Produkte ergibt, es sei denn, NetApp hat dem ausdrücklich in schriftlicher Form zugestimmt. Die Verwendung oder der Erwerb dieses Produkts stellt keine Lizenzierung im Rahmen eines Patentrechts, Markenrechts oder eines anderen Rechts an geistigem Eigentum von NetApp dar.

Das in diesem Dokument beschriebene Produkt kann durch ein oder mehrere US-amerikanische Patente, ausländische Patente oder anhängige Patentanmeldungen geschützt sein.

ERLÄUTERUNG ZU „RESTRICTED RIGHTS“: Nutzung, Vervielfältigung oder Offenlegung durch die US-Regierung unterliegt den Einschränkungen gemäß Unterabschnitt (b)(3) der Klausel „Rights in Technical Data – Noncommercial Items“ in DFARS 252.227-7013 (Februar 2014) und FAR 52.227-19 (Dezember 2007).

Die hierin enthaltenen Daten beziehen sich auf ein kommerzielles Produkt und/oder einen kommerziellen Service (wie in FAR 2.101 definiert) und sind Eigentum von NetApp, Inc. Alle technischen Daten und die Computersoftware von NetApp, die unter diesem Vertrag bereitgestellt werden, sind gewerblicher Natur und wurden ausschließlich unter Verwendung privater Mittel entwickelt. Die US-Regierung besitzt eine nicht ausschließliche, nicht übertragbare, nicht unterlizenzierbare, weltweite, limitierte unwiderrufliche Lizenz zur Nutzung der Daten nur in Verbindung mit und zur Unterstützung des Vertrags der US-Regierung, unter dem die Daten bereitgestellt wurden. Sofern in den vorliegenden Bedingungen nicht anders angegeben, dürfen die Daten ohne vorherige schriftliche Genehmigung von NetApp, Inc. nicht verwendet, offengelegt, vervielfältigt, geändert, aufgeführt oder angezeigt werden. Die Lizenzrechte der US-Regierung für das US-Verteidigungsministerium sind auf die in DFARS-Klausel 252.227-7015(b) (Februar 2014) genannten Rechte beschränkt.

Markeninformationen

NETAPP, das NETAPP Logo und die unter http://www.netapp.com/TM aufgeführten Marken sind Marken von NetApp, Inc. Andere Firmen und Produktnamen können Marken der jeweiligen Eigentümer sein.